



Desarrollo de plataformas móviles.

PARCIAL 1

La clínica privada "MediCare+" necesita modernizar su sistema de gestión de pacientes y turnos. Se desarrollarán dos productos: una PWA para el personal administrativo de escritorio, y una app móvil en Ionic para los médicos que hacen visitas a domicilio. Ninguna tiene backend real: toda la información se almacena en localStorage y la autenticación es simulada con usuarios fijos.

EJERCICIO 1 — PWA: MediCare+ Administración Web

Desarrollá una Progressive Web App en React orientada al personal administrativo de la clínica. Permite gestionar pacientes, asignarles turnos y armar un resumen de atención diaria.

Requerimientos:

- a) **LoginForm:** Tiene inputs de email y contraseña con estado local. Muestra el error *"Usuario o contraseña incorrectos"* inline (no alert) si las credenciales no coinciden.
- b) **Sesión persistente:** App guarda el usuario en localStorage. Al montar, si existe la clave, restaura la sesión directamente. El logout limpia esa clave y resetea el estado a null.
- c) **Avatar con previsualización:** en el componente **PerfilUsuario** (accesible desde el header). Mostralo como `<img>` circular en el header. Si no hay avatar, mostrar las iniciales del nombre en un círculo de color.
- d) **Control de acceso por rol:** el componente **Dashboard** recibe el usuario como prop y renderiza condicionalmente: si el rol es **"recepcionista"** no muestra la sección de estadísticas; si es **"medico"** no muestra el formulario de alta de pacientes. El control se hace con renderizado condicional, no con navegación.

Parte B — Gestión de pacientes

Cada paciente se almacenan en localStorage bajo "medicare_pacientes".

Requerimientos:



- a) **FormularioPaciente — alta y edición:** el mismo componente opera en dos modos según la prop **pacienteAEditar** (null = modo alta, objeto = modo edición). En modo alta todos los campos están vacíos; en modo edición se pre-cargan. Al guardar llama a **onGuardar(paciente)** del padre. Validaciones: nombre, apellido y DNI son obligatorios; el DNI debe tener entre 7 y 8 caracteres numéricos (validar con estado local, no con HTML5 required).
- b) **TablaPacientes:** recibe el array de pacientes y los callbacks **onEditar(paciente)** y **onEliminar(id)**. Renderiza una tabla HTML con una fila por paciente mostrando nombre completo, DNI y teléfono. El botón eliminar abre una modal de confirmación. El botón editar llama a onEditar y el padre se encarga de pasar el paciente al formulario.
- c) **BuscadorPacientes:** un input de búsqueda en el componente **Dashboard** filtra la lista visible por nombre, apellido o DNI simultáneamente (case-insensitive). El texto de búsqueda vive en Dashboard. La lista filtrada se pasa como prop a TablaPacientes. Justificá por qué el estado de búsqueda vive en Dashboard y no en TablaPacientes.

Parte C — Configuración PWA

- a) **manifest.json:** configurá name "*MediCare+ Admin*", short_name "*MediCare*", display "**standalone**", start_url "/", theme_color "**#0C2340**", background_color "**#FFFFFF**". Incluí íconos de 192x192 y 512x512. El tag **<link rel="manifest">** debe estar en el **index.html**.
- b) **Service Worker:** registrá el service worker (CRA: cambiar unregister por register / Vite: plugin PWA). Agregá un comentario en el código explicando qué significa la estrategia "**cache first**" y en qué casos conviene usarla para una app médica.

EJERCICIO 2 — Ionic React: MediCare+ Visitas Médicas

El médico inicia sesión, ve su agenda de visitas del día, selecciona una visita para registrar la atención, agrega medicamentos a una receta (carrito de prescripciones) y finaliza la visita. Todo se persiste en localStorage. La app debe sentirse clínica, rápida y confiable.

Parte A — Setup, navegación anidada y login

Requerimientos:



- a) **Estructura de navegación:** la app usa **ionTabs** con tres tabs principales (visibles post-login). Dentro de la tab de Visitas, hay una navegación anidada con **ionRouterOutlet** que permite ir de la lista de visitas al detalle de una visita sin perder la tab bar:

Tabs (post-login):

Tab 1 /visitas → VisitasPage (lista del día)

Tab 1 /visitas/:id → DetalleVisitaPage (detalle + receta)

Tab 2 /pacientes → MisPacientesPage

Tab 3 /perfil → PerfilMedicoPage

Pre-login:

/login → LoginPage

- b) **LoginPage:** mismas credenciales mockeadas del Ejercicio 1. Usá **ionInput** con **type="password"** y un botón de mostrar/ocultar contraseña que alterna el tipo entre **"password"** y **"text"** con estado local. Al fallar: **ionToast** de error. Al cargar: **ionLoading** de 1.5 segundos simulando verificación de credenciales.
- c) **Avatar en perfil:** igual que en la PWA, pero usando **ionAvatar** e **ionimg** de Ionic. Si no hay foto, mostrar las iniciales dentro de un **ionAvatar**.

Parte B — VisitasPage: agenda del día con estados

La visita es el objeto central de la app móvil:

Requerimientos:

- a) **ionList con ionItemSliding:** cada visita se muestra en un **ionItemSliding**. Deslizar a la izquierda muestra dos opciones: **"En camino"** (color primary, cambia estado a **en_camino**) y **"Cancelar"** (color danger, abre **ionAlert** de confirmación con campo de texto para el motivo). Deslizar a la derecha muestra **"Ver detalle"** (color secondary, navega a /visitas/:id).
- b) **ionReorderGroup:** el médico puede reordenar las visitas pendientes del día arrastrándolas. Usá **ionReorderGroup** con **disabled={false}** y manejá el evento **onionItemReorder** para actualizar el array con **event.detail.complete(items)**. Solo las visitas con estado **"pendiente"** son reordenables; las finalizadas o canceladas aparecen fijas al final.
- c) **ionSegment + badges dinámicos:** un **ionSegment** filtra por estado (Todas / Pendientes / En curso / Finalizadas). El tab bar de la app muestra un **ionBadge**



sobre el ícono de Visitas con la cantidad de visitas pendientes del día. Este badge se actualiza reactivamente cada vez que cambia el array de visitas en App. Se pasa como prop numérica desde App hacia el componente de tabs.

Entrega:

La entrega deberá ser realizada en git en una nueva rama.

Deberá agregar pantallazos de las apps funcionales dentro del archivo readme.md